

OpenMythos

Guía técnica de replicación, validación y puesta en marcha

Documento técnico independiente basado en el repositorio público OpenMythos, la documentación del proyecto y la literatura académica citada por el propio repositorio.
Fecha: 19 de abril de 2026

Qué contiene Arquitectura, ejecución mínima, limitaciones, parche recomendado y hoja de ruta.	Qué no promete No demuestra que Claude Mythos real use esta arquitectura ni aporta pesos entrenados.	Qué sí valida Que el repositorio actual arranca, ejecuta inferencia mínima y puede auditarse técnicamente.
---	--	--

Arquitectura operativa de OpenMythos

Síntesis del flujo realmente implementado en el repositorio público

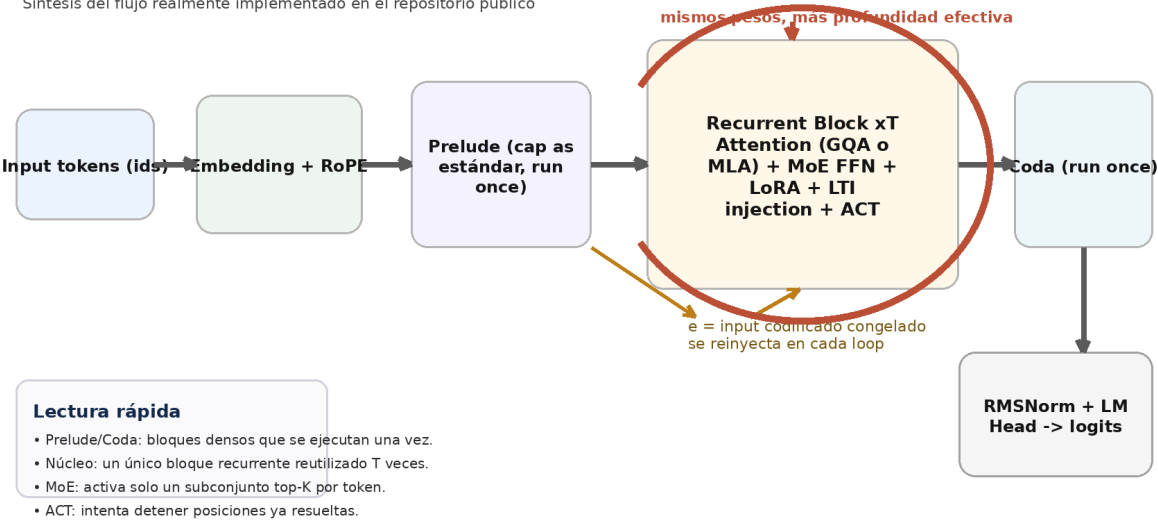


Figura 1. Resumen visual del flujo operativo implementado en el repositorio.

Aviso clave. OpenMythos se presenta explícitamente como una reconstrucción teórica independiente, no afiliada a Anthropic y basada en investigación pública y especulación razonada.

1. Resumen ejecutivo

OpenMythos es una implementación open-source en PyTorch que intenta materializar una hipótesis concreta: que “Claude Mythos” podría parecerse a un Recurrent-Depth Transformer con un bloque compartido en bucle, mezcla de expertos en el núcleo recurrente, atención conmutables GQA/MLA, inyección LTI estable y halting ACT. Lo importante es separar tres planos: la implementación existe y es auditable; la arquitectura es coherente con literatura reciente; pero la equivalencia con cualquier sistema propietario de Anthropic no está demostrada.

- Sí puedes replicar el repositorio actual, ejecutarlo y usarlo como base de investigación.
- No puedes tratarlo como una filtración ni como una réplica confirmada de Claude Mythos.
- La vía correcta es usarlo como banco de pruebas para transformers en bucle, razonamiento latente e inferencia con profundidad variable.
- Para “hacerlo funcionar” de forma robusta conviene añadir: empaquetado, entrenamiento reproducible, tokenizer, scripts de benchmark y un pequeño parche numérico en LTIInjection.

2. Qué es realmente OpenMythos

El README del repositorio lo define de forma explícita como una reconstrucción teórica independiente, basada en literatura pública y no afiliada a Anthropic. La propia estructura del proyecto muestra tres etapas claras: Prelude, Recurrent Block y Coda; atención conmutable entre MLA y GQA; y un FFN MoE esparso dentro del bloque recurrente. La documentación adicional expone los campos de configuración, el flujo de forward y el mecanismo de generación autoregresiva.

Elemento	Estado	Lectura técnica
Repositorio público	Sí	README, código fuente y documentación visibles.
Licencia MIT	Sí	El repositorio declara licencia MIT.
Implementación PyTorch	Sí	El modelo y los submódulos están escritos en PyTorch.
Ejemplo mínimo de uso	Sí	example.py crea el modelo, hace forward y generación.
Pruebas automatizadas	Sí	Hay suite pytest relativamente amplia.
Pesos preentrenados	No	El repo no publica checkpoints entrenados.
Tokenizer propio	No visible	No aparece tokenizer ni vocabulario dedicados en el árbol mostrado.
Pipeline de entrenamiento completo	No	No se ve script de entrenamiento, datasets ni recetas de escalado.
Afirmación de equivalencia con Claude Mythos	No	El propio proyecto se define como reconstrucción teórica.

Archivo	Propósito
open_mythos/main.py	Implementación central del modelo, submódulos y generación.
docs/open_mythos.md	Referencia funcional y de configuración.
example.py	Arranque mínimo reproducible.
test_main.py	Validación de formas, caché KV, MoE, LoRA, ACT, LTI, generación.
requirements.txt	Dependencias mínimas: torch y pytest.

Fuentes base usadas en esta guía

- [R1] Repositorio OpenMythos (README) - GitHub.
- [R2] Documentación open_mythos.md - GitHub.
- [R3] Hilo de lanzamiento de Kye Gomez / Thread Reader.
- [R4] Loop, Think, & Generalize: Implicit Reasoning in Recurrent-Depth Transformers (arXiv:2604.07822).
- [R5] Parcae: Scaling Laws For Stable Looped Language Models (arXiv:2604.12946).
- [R6] Reasoning with Latent Thoughts: On the Power of Looped Transformers (arXiv:2502.17416).
- [R7] Training Large Language Models to Reason in a Continuous Latent Space / Coconut (arXiv:2412.06769).
- [R8] DeepSeek-V2 (MLA) (arXiv:2405.04434).
- [R9] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints (arXiv:2305.13245 / EMNLP 2023).
- [R10] DeepSeekMoE (arXiv:2401.06066).
- [R11] Universal Transformers (arXiv:1807.03819).
- [R12] Adaptive Computation Time for Recurrent Neural Networks (arXiv:1603.08983).

3. Intuición arquitectónica

La hipótesis central es que la profundidad útil no tiene por qué venir de apilar capas distintas. Puede venir de aplicar varias veces el mismo bloque compartido durante un único forward, reinyectando la representación codificada del input. Eso desplaza parte del escalado desde “más parámetros almacenados” hacia “más cómputo de inferencia sobre un mismo bloque”.

Razonamiento explícito vs razonamiento latente en bucle

Comparación conceptual usada por el proyecto para justificar la hipótesis RDT

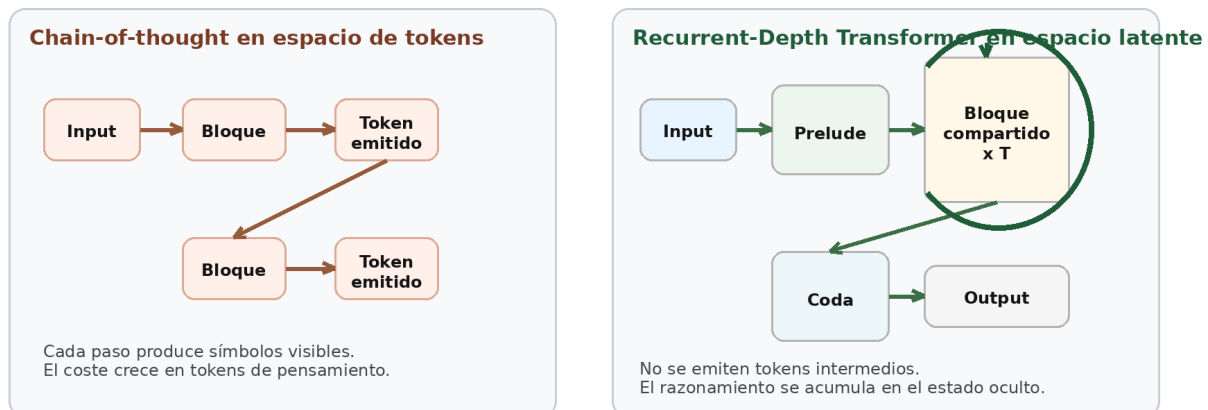


Figura 2. Comparación conceptual: chain-of-thought explícito frente a RDT latente en bucle.

En OpenMythos, el razonamiento intermedio no se expresa como tokens visibles. Se acumula en el estado oculto h_t , actualizado con la regla recurrente $h_{t+1} = A \cdot h_t + B \cdot e + \text{Transformer}(h_t, e)$. El proyecto presenta esa decisión como un intento de capturar razonamiento latente, conectándolo con trabajos recientes sobre looped transformers y continuous latent reasoning.

4. Despiece técnico por componente

4.1 Prelude y Coda

Son bloques transformer estándar ejecutados una sola vez. En ambos casos el FFN es denso tipo SwiGLU, no MoE. Su papel es preparar la representación de entrada y proyectar la salida final antes de RMSNorm + LM head.

4.2 Bloque recurrente compartido

Es el núcleo del diseño. El mismo TransformerBlock se reutiliza hasta `max_loop_iters` veces. Cada iteración recibe señal de profundidad por un embedding sinusoidal de índice de loop, puede modificar su comportamiento con un adaptador LoRA dependiente de la iteración y vuelve a mezclar la entrada congelada e mediante la inyección LTI.

4.3 Atención

- GQA: reduce caché KV compartiendo pares K/V entre grupos de cabezas de consulta.
- MLA: comprime la ruta KV en un latente de bajo rango y reconstruye K/V al vuelo; es la opción más ambiciosa de la implementación.

4.4 MoE en el recurrente

El recurrente usa routed experts top-K más shared experts siempre activos. La idea es heredar de DeepSeekMoE tanto la segmentación fina de expertos como el aislamiento de expertos compartidos.

4.5 Inyección LTI y ACT

La inyección LTI intenta mantener estabilidad espectral imponiendo A en (0,1). ACT, por su parte, aprende una probabilidad de halting por posición para que algunas posiciones dejen de acumular actualizaciones antes que otras.

5. Qué he validado de forma práctica

He descargado el código público relevante y lo he ejecutado localmente en CPU con PyTorch 2.10.0+cpu. Con configuraciones pequeñas se verificó arranque correcto, forward pass y generación mínima tanto con GQA como con MLA. Además, la suite pytest del proyecto existe y cubre componentes clave como RoPE, atención, MoE, LoRA, ACT, LTI, recurrente y generación.

Chequeo	Resultado observado
Import y construcción del modelo	Correcto.
Forward mínimo con GQA	Correcto.
Forward mínimo con MLA	Correcto.
Generación mínima autoregresiva	Correcta en pruebas reducidas.
Matriz A de LTI	Valores dentro de rango estable en pruebas normales.
Suite pytest completa sin tocar	Se detectó al menos un fallo de estabilidad bajo actualización extrema.

Hallazgo importante. La prueba `test_spectral_radius_stable_after_large_grad_step` falla porque `A.max()` puede redondear a 1.0 tras un paso de SGD con learning rate 1e3. El problema parece ser numérico/operacional, no conceptual: la parametrización sigue empujando A al rango estable, pero float32 puede devolver exactamente 1.0 en el límite.

6. Procedimiento mínimo para replicarlo hoy

El flujo recomendado es este, sin adornos y sin asumir nada que el repositorio todavía no ofrece.

6.1 Preparación del entorno

```
git clone https://github.com/kyegomez/OpenMythos.git
cd OpenMythos
python -m venv .venv
# Linux/macOS:
source .venv/bin/activate
# Windows PowerShell:
# .venv\Scripts\Activate.ps1
python -m pip install -U pip
pip install -r requirements.txt
```

6.2 Arranque rápido

```
python example.py

# o, si prefieres una verificación más compacta,
# usa el script adjunto a esta guía:
python openmythos_quickstart.py
```

6.3 Validación básica

```
pytest -q

# si quieres aislar el fallo numérico conocido:
pytest -q -k "not test_spectral_radius_stable_after_large_grad_step"
```

6.4 Qué debes esperar

- Que el modelo construya embeddings, RoPE, prelude, recurrente, coda y LM head sin error.
- Que el forward devuelva logits con forma (B, T, vocab_size).
- Que generate añada tokens y reutilice caché KV.
- Que la variante MLA sea más pesada conceptualmente, pero funcional en configuración pequeña.

7. Procedimiento para usarlo como banco de pruebas serio

El repositorio actual es suficiente para exploración arquitectónica, pero no para una campaña de entrenamiento o comparación seria. A continuación se muestra la ruta mínima para convertirlo en una base reproducible.

Fase	Objetivo	Acción recomendada
Fase 0	Congelar una versión	Haz fork, etiqueta commit y guarda un requirements-lock.
Fase 1	Corregir estabilidad operativa	Aplica el parche de LTI para evitar que A llegue exactamente a 1.0 en float32.
Fase 2	Añadir empaquetado	pyproject.toml, instalación editable y CLI simple.
Fase 3	Tokenizer y datos	Define tokenizer, dataset y formato causal LM.
Fase 4	Entrenamiento mínimo	Script train.py con warmup, clipping, checkpoints y logging.
Fase 5	Benchmarks	Curvas frente a nº de loops, comparación GQA vs MLA, y ablations MoE/LoRA/ACT.

La clave aquí es no intentar “entrenar a lo grande” en el primer paso. Primero hay que demostrar tres cosas: que el modelo sobreajusta un lote pequeño, que la pérdida baja de forma estable en un dataset de juguete y que aumentar `n_loops` en inferencia cambia el comportamiento de forma predecible.

8. Parche recomendado para LTIInjection

El cambio más pequeño y con mejor relación coste-beneficio es blindar numéricamente `get_A()` para que nunca devuelva exactamente 1.0. Eso alinea la implementación con la intención declarada por el propio código: mantener $\rho(A) < 1$ por construcción.

```
def get_A(self) -> torch.Tensor:
    A = torch.exp(-torch.exp((self.log_dt + self.log_A).clamp(-20, 20)))
    return torch.minimum(
        A,
        torch.nextafter(torch.ones_like(A), torch.zeros_like(A)),
    )
```

Este parche no cambia la idea arquitectónica. Solo evita que un valor en el borde, perfectamente estable en teoría, se convierta en 1.0 exacto por redondeo en float32 y rompa la aserción estricta de la prueba.

9. Script mínimo recomendado para esta guía

Adjunto con este documento un archivo `openmythos_quickstart.py`. Está pensado para ejecutarse desde la raíz del repositorio y producir una verificación compacta para GQA y MLA.

```
"""
openmythos_quickstart.py
Ejemplo mínimo y verificable para arrancar OpenMythos en CPU o GPU.

Uso esperado desde la raíz del repo clonado:
python openmythos_quickstart.py
"""

import time
import torch
from open_mythos.main import OpenMythos, MythosConfig

def run(attn_type: str) -> None:
    base = dict(
        vocab_size=256,
        dim=64 if attn_type == "mla" else 48,
        n_heads=4,
        n_kv_heads=4,
        max_seq_len=64,
        max_loop_iters=2,
        prelude_layers=1,
        coda_layers=1,
        n_experts=4,
        n_shared_experts=1,
        n_experts_per_tok=1,
        expert_dim=16,
        lora_rank=4,
        attn_type=attn_type,
    )

    if attn_type == "mla":
        cfg = MythosConfig(
            **base,
            kv_lora_rank=8,
            q_lora_rank=16,
            qk_rope_head_dim=8,
            qk_nope_head_dim=8,
            v_head_dim=8,
        )
```

```

else:
    cfg = MythosConfig(**base)

model = OpenMythos(cfg)
model.eval()

total = sum(p.numel() for p in model.parameters())
ids = torch.randint(0, cfg.vocab_size, (1, 8))

t0 = time.time()
with torch.no_grad():
    logits = model(ids, n_loops=2)
    generated = model.generate(ids, max_new_tokens=2, n_loops=2)
    A = model.recurrent.injection.get_A().detach()
dt = time.time() - t0

print(f"\n[{attn_type.upper()}]")
print(f"Parámetros: {total:,}")
print(f"Forma logits: {tuple(logits.shape)}")
print(f"Forma generación: {tuple(generated.shape)}")
print(f"Rango A discreta: min={A.min().item():.8f}, max={A.max().item():.8f}")
print(f"Tiempo total: {dt:.3f}s")

if __name__ == "__main__":
    print("Iniciando verificación mínima de OpenMythos...")
    run("gqa")
    run("mla")
    print("\nListo.")

```

10. Qué falta para afirmar que “funciona” de verdad

- Pesos entrenados reproducibles y checkpoints públicos.
- Tokenizer y preparación de datos definidos.
- Script de entrenamiento completo con configuración reproducible.
- Benchmarks controlados que demuestren ganancia real al aumentar loops.
- Comparativas contra un transformer denso de igual presupuesto de FLOPs y/o parámetros.
- Experimentos que midan overthinking, saturación de loops y estabilidad del halting.

Sin esas piezas, OpenMythos funciona como artefacto de investigación y prototipado arquitectónico. No funciona aún como modelo fundacional reproducible comparable a un LLM completo.

11. Recomendación estratégica

La forma más inteligente de aprovechar este proyecto no es venderlo como réplica cerrada de un modelo propietario, sino convertirlo en una plataforma de experimentación rigurosa para tres preguntas: (1) cuánto razonamiento extra aporta el looping en inferencia; (2) cuándo MLA merece la complejidad añadida frente a GQA; y (3) si MoE dentro de un bloque recurrente realmente mejora la relación rendimiento/cómputo.

- Para prototipos rápidos: empieza con GQA, pocos loops y sin obsesionarte con MLA.
- Para investigación de memoria: usa MLA solo después de fijar el resto del stack.
- Para análisis de estabilidad: instrumenta A, halting probabilities y distribución de expertos por loop.

12. Conclusión

OpenMythos merece atención no porque demuestre qué hay dentro de Claude Mythos, sino porque empaqueta en un repositorio pequeño varias líneas de investigación muy actuales: transformers en bucle, razonamiento latente, profundidad de inferencia adaptable, MoE y compresión de KV cache. Como base reproducible es prometedora; como réplica confirmada, no. La mejor forma de extraer valor hoy es

correrlo, parchear la estabilidad operacional, añadir scripts de entrenamiento y convertirlo en un entorno de experimentación controlada.

Apéndice A. Referencias bibliográficas y enlaces

- [R1] OpenMythos README - GitHub. <https://github.com/kyegomez/OpenMythos>
- [R2] OpenMythos docs/open_mythos.md - GitHub.
https://github.com/kyegomez/OpenMythos/blob/main/docs/open_mythos.md
- [R3] Thread Reader del hilo de lanzamiento de Kye Gomez.
<https://threadreaderapp.com/user/KyeGomezB>
- [R4] Loop, Think, & Generalize: Implicit Reasoning in Recurrent-Depth Transformers.
<https://arxiv.org/abs/2604.07822>
- [R5] Parcae: Scaling Laws For Stable Looped Language Models. <https://arxiv.org/abs/2604.12946>
- [R6] Reasoning with Latent Thoughts: On the Power of Looped Transformers.
<https://arxiv.org/abs/2502.17416>
- [R7] Training Large Language Models to Reason in a Continuous Latent Space (Coconut).
<https://arxiv.org/abs/2412.06769>
- [R8] DeepSeek-V2. <https://arxiv.org/abs/2405.04434>
- [R9] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.
<https://arxiv.org/abs/2305.13245>
- [R10] DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models.
<https://arxiv.org/abs/2401.06066>
- [R11] Universal Transformers. <https://arxiv.org/abs/1807.03819>
- [R12] Adaptive Computation Time for Recurrent Neural Networks. <https://arxiv.org/abs/1603.08983>

Apéndice B. Archivos adjuntos

- openmythos_quickstart.py - verificación mínima ejecutable.
- openmythos_lti_patch.diff - parche sugerido para la estabilidad operacional de LTIInjection.